Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Michael Catania Jr	mcatania@udel.edu
Friend (optional)	N/A	N/A

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

My program will take inputs from the user, including a key and a mood, and produce a chord progression that matches their selections, along with guitar fingering diagrams for each chord.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

https://chordcraft.org/ addressed this problem in a similar way I proposed. The main difference is that this website gives chord diagrams only for piano, while my webpage would give guitar diagrams. *https://autochords.com/ also attempts to address this similarly, though it also lacks any diagrams.*

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

This web application could be used by singer/songwriters who need fresh ideas for a song

centered around a particular mood. It can also be utilized as a learning and practice tool by beginner guitarists who are trying to learn which chords work well together in progression.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

My program will take a key, mood, progression length, and 7th-chord choice from the user and use them to build a fitting chord progression, along with the parent scale and guitar fingering diagrams for each chord.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

The user will provide a key, a mood (Joyful, Peaceful, Sorrowful, Mysterious, or Bluesy), a progression length, and whether to include 7th chords. The user can also type a name for a progression when saving it.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

The program will convert the selected key into its parent scale using a list of the twelve notes and a pattern of whole and half steps. It will use the mood to choose a pattern of Roman numerals (e.g. I-V-vi-IV for Joyful) and use an if statement to adjust the pattern for the chosen length. Each numeral will be converted into a chord by finding the matching note in the scale, and an if statement will add 7ths if the user chooses. The program will look up fret positions in a dictionary of guitar fingerings for each chord. Minor keys will be treated as their relative major key. Saved progressions will be stored in a list, and the transposition button will shift the key up or down, rebuilding the progression accordingly.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user will get back the progression pattern, the parent scale, the chord progression itself, and a fingering diagram for each chord. The user can also save a progression and view their saved progressions.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Chord, progression, key, mood, length, saved progressions.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

The most important thing I listed is the chord, since every progression is made of chords and every diagram is drawn from one.
Details I need: Chord name, root note, chord type, and the fret position for each of the six strings.
Details I will ignore: Which fingers are used on each fret, alternate voicings and inversions, sound qualities such as tone and volume, strumming and rhythm, and tuning variations.

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

Dealing with minor keys (mapping to relative major), transposition, and creating professional-looking chord diagrams.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow

learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will make sure the LLM knows the project constraints and the tools available to accomplish it. I will read through all the code the LLM generates in order to understand its purpose and how it accomplishes its function.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

I want to create a tool I can use for my own musical development that is simple to use and navigate.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to build a simple web tool to help beginner guitarists or songwriters come up with chord progressions. What are 3 common problems people face when trying to come up with a progression, and what are 3 simple features a basic tool should have?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The AI suggested that three common problems beginners face are:

1. Not knowing which chords work together

2. Getting stuck in repetitive progressions

3. Not knowing how a progression will sound or how to play it

It stated that the underlying issue is the gap between music theory knowledge and actually playing music.

3 basic features it suggested were:

1. Chord progression generator

2. Chord diagrams and audio playback

3. Easy customization and regeneration

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

The playback feature you suggested is too complicated for an introductory programming project since I would need to program the sound of all the possible chords in all orders. What is a simpler alternative?

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

I like the idea about interactive chord diagrams. I could even link the chord diagrams to a page listing different voicings of the same chord. I am more focused now on making the webpage easy to understand especially for beginner guitarists.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I will use the drafter guide, reference page, and glossary. I will also use LLM's to explain how the code works. And I will utilize TA help when the process becomes too confusing.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will analyze each function on its own with unit tests and then look at the big picture to see how the functions work together with proper python and Drafter syntax.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Prompt:

Help me brainstorm an idea I have for my Computer Science final project. Attached are the requirements. I want to create a program that will take inputs of simple keys, moods, and other selections from a user in order to produce chord progressions that match their selections along with fingering diagrams for guitar. Possibly also a button to transpose the progressions.

Reply:
This idea fits the requirements well. Here's how I'd shape it.

Core concept

A user picks a key, a mood, and a few options, and the app generates a chord progression. Each chord gets a guitar fingering diagram. A transpose button shifts the whole progression.

Pages (need 5)

1. ***Home/Setup: pick key, mood, and options.***
2. ***Result: the progression with a diagram for each chord.***
3. ***Chord detail: one chord up close, with its fingering and notes.***
4. ***Saved progressions: a list of what the user has kept.***
5. ***About/Help: explains the moods and how progressions are built.***

Routes (need 7)

***`index`**, **`generate`**, **`show_result`**, **`transpose`**, **`save_progression`**, **`view_saved`**, **`chord_detail`**. You could add **`delete_saved`** or **`regenerate`** for extra margin.*

Dataclasses

- *Chord (second dataclass): **`name`**, **`root`**, **`quality`** (major/minor/7th), **`fingering`** (e.g. a list of fret numbers per string), **`notes`**.*
- *Progression: a list of Chord instances plus a **`name`**/**`mood`**. You can skip this one if it feels like too much, but it makes saving cleaner.*
- *State (needs 4+ fields, and 4 must be meaningfully modified in routes):*
  - ***`key: str`** changed by the generate and transpose routes*
  - ***`mood: str`** changed by the generate route*
  - ***`current_progression: list[Chord]`** changed by the generate, transpose, and regenerate routes*
  - ***`saved_progressions: list[Progression]`** changed by the save and delete routes*
  - *Optional: **`capo_fret: int`** or **`use_sevenths: bool`***

Inputs (need 3+)

- ***`SelectBox`** for key (C, G, D, A, E, ...)*
- ***`SelectBox`** for mood (happy, sad, dreamy, tense, bluesy)*
- ***`CheckBox`** for "include 7th chords" or "use only open chords"*
- ***`TextBox`** for naming a saved progression*
- *Optional **`SelectBox`** for progression length (4 or 8 chords)*

How the logic works

The best approach is Roman numeral templates. Each mood maps to a list of progressions written as scale degrees:

- *Happy: I–V–vi–IV*
- *Sad: vi–IV–I–V or i–VI–III–VII*
- *Bluesy: I–IV–I–V*
- *Dreamy: I–iii–IV–V*

Your program converts numerals into actual chords for the chosen key (in C, V is G). Transposing is then just changing the key and recomputing. That makes

transposition simple and unit-testable, and it's a good fit for your rubric.

Meeting the rubric items

- *If statements: major vs. minor mood handling, "open chords only" filtering, empty-saved-list check, 7th chord option.*
- *Loop over class instances: iterating through `current_progression` to render each chord's diagram, or through `saved_progressions` to list them.*
- *Unit tests: the numeral-to-chord conversion, transposition, and chord lookup are pure functions, so each is easy to test.*

The hard part: fingering diagrams

Three options, from easiest to most impressive:

1. *Lookup dictionary: store fingerings for common chords (e.g. `"G": [3, 2, 0, 0, 0, 3]`). Simple, but you need to limit which chords you support.*
2. *Text/ASCII diagrams generated from the fret lists, displayed in a monospace block.*
3. *Generated image/SVG drawn from the fret data. This looks best, but check whether Drafter supports embedding images this way. If not, a table of cells could work.*

I'd start with option 1 plus 2, then upgrade if time allows. Note that transposing to keys outside your lookup table is the catch: either restrict keys to ones with open-chord shapes, or add barre chord shapes (E-shape and A-shape patterns that slide up the neck), which would make for an impressive "advanced technique" to document.

Stretch ideas

- *"Surprise me" random mood*
- *Capo suggestions that make a hard key easier*
- *Relative minor/major toggle*
- *Difficulty filter (beginner, intermediate)*

Questions to decide on

1. *Should you support minor keys, or only major keys at first?*
2. *Open chords only, or barre chords too?*
3. *How many moods do you want? Four or five is plenty.*

Want me to sketch the State and dataclass code next, or draft the planning document

for Week 11?

Prompt:

I can create a lookup dictionary for common chords.
I could support minor keys by making them synonymous with their corresponding major key.
I would be ok supporting barre chords in the dictionary.
I think 5 moods could work: Joyful, Peaceful, Sorrowful, Mysterious, etc.

Help me plan meeting all the requirements for this and provide any ideas you have.

Reply:

Those choices all simplify things. Here's a plan that maps your design onto each requirement.

Design decisions to lock in

Minor keys as relative majors*: Since Am and C share the same chords, the user picks "A minor" and you internally treat it as C major. One thing to handle: the tonic is different. A progression in A minor should start on Am, not C. So store the key as a major key plus a* `is_minor` *flag (or a* `tonic_offset`*), and let the Roman numeral templates for minor moods start from vi. Otherwise "A minor" just gives the same output as C major.*

Fifth mood*: Joyful, Peaceful, Sorrowful, Mysterious, and then something like* ***Energetic*** *(driving, rock-style) or* ***Nostalgic*** *or* ***Bluesy****. Bluesy is the most distinct musically (I–IV–V with 7ths).*

Data design

```
@dataclass
class Chord:
    name: str           # "Am7"
    root: str           # "A"
    quality: str        # "minor7"
    frets: list[int]    # [-1, 0, 2, 0, 1, 0]  (-1 = muted)
    is_barre: bool

@dataclass
class Progression:
    name: str
    mood: str
    key: str
```

```
    chords: list[Chord]

@dataclass
class State:
    key: str
    mood: str
    current_progression: list[Chord]
    saved_progressions: list[Progression]
    use_sevenths: bool
    transpose_steps: int
```

Requirement checklist

Requirement	How you meet it
7+ routes	`index`, `generate`, `show_result`, `transpose`, `save_progression`, `view_saved`, `chord_detail`, `delete_saved` (8 gives you a spare)
5+ pages	Home, Result, Chord Detail, Saved, About/Mood Guide
State with 4+ fields	6 fields above
List of class instances + 2nd dataclass	`current_progression: list[Chord]`, `saved_progressions: list[Progression]`
4 fields modified, non-constant	`key` and `mood` (generate), `current_progression` (generate/transpose), `saved_progressions` (save/delete), `transpose_steps` (transpose). That's five, so you're safe even if one is questioned.
3+ inputs	SelectBox (key), SelectBox (mood), CheckBox (sevenths), TextBox (progression name), optional SelectBox (length)
3+ meaningful ifs	Minor vs. major handling, add 7ths if checked, empty saved list message, duplicate-name check on save, chord missing from dictionary fallback

Loop over instance attribute	*Loop through `current_progression` to render each chord's `frets`; loop through `saved_progressions` to list them*
No globals	*Constants only (the chord dictionary, mood templates, and note list can all be uppercase constants)*
Unit tests per route	*Test the pure helper functions heavily, then route tests that check returned page content*
Advanced techniques documented	*Transposition via modular arithmetic on a 12-note list, and barre shape sliding (if you do it). Comment these thoroughly.*

The core logic (keep it in helper functions)

1. *`get_scale(key) -> list[str]`: the seven notes of the major key.*
2. *`numeral_to_chord(numeral, key, use_sevenths) -> Chord`: converts "vi" to "Am" in C.*
3. *`build_progression(key, mood, use_sevenths) -> list[Chord]`: picks a template for the mood and converts each numeral.*
4. *`transpose_chord(chord, steps) -> Chord`: shifts root by semitones using `(index + steps) % 12` over a list of 12 note names.*
5. *`lookup_fingering(name) -> list[int]`: dictionary lookup.*

These are all testable without Drafter, which helps your unit-testing requirement and keeps functions short.

Transposition: a design choice

Two ways to do it:

- ***Recompute from the key**: simplest, always correct, and uses your lookup dictionary.*
- ***Shift the chord roots directly**: more "advanced," but you'd need fingerings for every resulting chord.*

I'd go with recompute. Your lookup dictionary needs to cover all 12 roots × a few qualities (major, minor, 7th, minor 7th, maybe diminished). That's about 60 entries. Barre chords make this feasible: you only need two or three shape templates (E-shape, A-shape) plus open chords, and you can generate the rest by adding the fret offset. That generation function would be your documented advanced technique, and it saves you typing 60 entries

by hand.

Displaying diagrams

Text diagrams in a `<pre>`-style block are the safest start. For example, a six-string grid with an X for muted, O for open, and a dot for fretted. Check whether Drafter's `Div`/`PreformattedText` components let you do monospace. If not, an HTML table built from components, with a cell per string/fret, also works. Build the text version first, and treat anything prettier as polish.

Ideas to make it impressive

- ***Strumming pattern suggestion** per mood (text only, low effort).*
- ***Difficulty filter**: "beginner mode" checkbox that restricts to open chords and swaps in easier substitutes.*
- ***Capo suggestion**: if the progression has many barre chords, suggest a capo position to make it easier.*
- ***Mood guide page** explaining why each mood sounds the way it does (major vs. minor, tension from 7ths), which doubles as your About page.*
- ***Randomization** among 2-3 templates per mood so repeated clicks give variety. Note that randomized routes are exempt from unit tests, but keep the random choice inside a helper so the rest stays testable.*
- ***Chord detail page** that also shows notes in the chord and a "play along" tip.*

Suggested timeline

- ***Week 11 (Design)**: finalize the chord dictionary format, mood templates, page sketches, and planning document.*
- ***Week 12 (Development)**: build helpers and tests first, then routes, then pages, then deploy.*
- ***Week 13-14**: user testing (watch whether people understand the minor key handling and the transpose button), then fix.*

Want me to draft the planning document for the repository, or start with the chord dictionary and mood template data structures?

Prompt:

I like the blues idea. How could I deal with sharp and flat keys and transposition?

Reply:

Bluesy is a good fifth mood. The 12-bar blues (I–I–I–I–IV–IV–I–I–V–IV–I–I, usually with dominant 7ths) gives you a distinctive template, and it's a good use of your "use 7ths" logic

since blues chords are I7, IV7, V7.

The core problem

Music has 12 pitches but 7 letter names, and the same pitch can be spelled two ways (C♯ = D♭). You need a consistent way to name notes so your lookup dictionary always finds a match.

Recommended approach: pick one spelling and stick to it

Use a single 12-note list with sharps for black keys:

```
NOTES = ["C", "C#", "D", "D#", "E", "F", "F#", "G", "G#", "A", "A#",
"B"]
```

Everything internal (transposing, building scales, dictionary keys) uses these names. Then transposition is just index math:

```
def transpose_note(note: str, steps: int) -> str:
    """Shift a note name by a number of semitones."""
    return NOTES[(NOTES.index(note) + steps) % 12]
```

The `% 12` handles wrapping around, so B up one step gives C.

Building scales without worrying about sharps/flats

Define the major scale as a pattern of semitone offsets from the root instead of memorizing each key:

```
MAJOR_SCALE_STEPS = [0, 2, 4, 5, 7, 9, 11]

def get_scale(key: str) -> list[str]:
    """Return the seven notes of a major key."""
    root_index = NOTES.index(key)
    return [NOTES[(root_index + step) % 12] for step in
MAJOR_SCALE_STEPS]
```

Then a Roman numeral just indexes into the scale: I is `scale[0]`, V is `scale[4]`, vi is `scale[5]`. The same function works for every key, so you don't need separate code for sharp or flat keys.

Handling flat names the user expects

Guitarists say "B♭" and "E♭", not "A#" and "D#". Two small additions fix this:

1. ***Input side**: let the SelectBox show friendly names, and convert to your internal names with a small dictionary:*

```
FLAT_TO_SHARP = {"Db": "C#", "Eb": "D#", "Gb": "F#", "Ab": "G#", "Bb": "A#"}
```

2. ***Output side**: when displaying, convert back using a "preferred display name" dictionary, usually flats for D♯, G♯, and A♯ (Eb, Ab, Bb) and sharps for F♯ and C♯. That matches how guitarists usually read keys.*

Keep the conversion at the edges (input and display) and use one spelling in the middle. That keeps your logic simple and your tests easy.

The honest caveat

Strictly speaking, in the key of F, the fourth note is B♭, not A♯. A music theorist would call your approach "enharmonic equivalence," and it's fine for chord names on a guitar app since the fingerings are identical. But if you display the scale degrees or notes in a chord, convert to display names so users see Bb, not A#. You can mention this as a documented simplification in your code.

Minor keys

Since you're treating minor as the relative major: A minor's relative major is C, which is 3 semitones up. A tiny dictionary or formula does it:

```
def relative_major(minor_root: str) -> str:
    """Return the relative major key of a minor key."""
    return transpose_note(minor_root, 3)
```

Then minor-mood templates start on the vi chord, which lands back on the minor tonic.

Transposition options

- ***Semitone buttons** ("Up ½ step" / "Down ½ step"): `transpose_steps` in State changes by ±1, and you regenerate from the new key. Simple and tracks well in your State.*
- ***Jump to a key**: a SelectBox lets the user pick any key directly.*
- ***Progression transpose**: since your progression is built from numerals, transposing*

means changing the key and rebuilding. Your Chord objects never need to be shifted individually.

Fingerings for sharps and flats

Your dictionary keys should use your internal names (`"A#"`, `"A#m"`, `"A#7"`). With barre shapes this is easy to generate rather than type out:

- ***E-shape barre** (root on the 6th string): fret number = offset from E. For F (1 semitone above E), fret 1. For A#, fret 6.*
- ***A-shape barre** (root on the 5th string): fret number = offset from A. For B, fret 2. For C#, fret 4.*

So a function like `barre_fingering(root, shape)` can compute the frets by adding the offset to the shape's base pattern. Prefer open chords where they exist (C, A, G, E, D, Am, Em, Dm, and so on) and fall back on barre shapes for everything else. Mark those with `is_barre=True`.

Tests worth writing

- *`transpose_note("B", 1) == "C"` (wrap-around)*
- *`transpose_note("C", -1) == "B"` (negative steps)*
- *`get_scale("C") == ["C","D","E","F","G","A","B"]`*
- *`get_scale("F")` includes `"A#"` internally*
- *`relative_major("A") == "C"`*
- *A barre fingering test: F should be `[1, 3, 3, 2, 1, 1]`*

Python handles negative modulo correctly (`-1 % 12 == 11`), so down-transposition works without special code.

Want me to sketch the full chord lookup structure next, including how to generate the barre chords from shape templates?

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.